

University of Wah

Department of Computer Science



Submit to:

Ms. Muniba Khan

Submit by:

Muhammad Talha

Course:

DSA Lab

Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Task (1)

```
int main(){
    Node* root = NULL;
    int values[10];

    cout<<"Enter 10 Number: ";
    for (int i = 0; i < 10; i++)
    {
        cin>>values[i];
    }
    cout<<"Values: ";
    for (int i = 0; i < 10; i++)
    {
        cout<<values[i]<<" ";
    }

    for (int value: values)
    {
        root = insert(root, value);
    }
```

OutPut

```
Enter 10 Number: 12 32 43 54 65 76 87 13 54 65
Values: 12 32 43 54 65 76 87 13 54 65 [1] + Done
```

```

struct Node
{
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int value){
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

Node* insert(Node* root, int value){
    if(root == NULL){
        return createNode(value);
    }

    if(value < root->data){
        root->left = insert(root->left, value);
    }
    else if(value > root->data){
        root->right = insert(root->right, value);
    }

    return root;
}

```

(2)
Inorder
Code

```

}
cout << "In-order Traversal: ";
inorder(root);

```

```

void inorder(Node* root){
    if(root != NULL){
        inorder(root->left);
        cout<<root->data<<" - ";
        inorder(root->right);
    }
}

```

Output

```

In-order Traversal: 20 - 30 - 40 - 50 - 60 - 70 - 80 - [1] + Done

```

PreOrder and PostOrder

Code

```
void preorder(Node* root){
    if(root != NULL){
        cout << root->data << " - ";
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root){
    if(root != NULL){
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " - ";
    }
}
```

Output

```
In-order Traversal: 20 - 30 - 40 - 50 - 60 - 70 - 80 -
Pre-order Traversal: 50 - 30 - 20 - 40 - 70 - 60 - 80 -
Post-order Traversal: 20 - 40 - 30 - 60 - 80 - 70 - 50 -
```

(3)

```
int findmin(Node* root){
    while (root->left != NULL)
    {
        root = root->left;
    }
    return root->data;
}

int findmax(Node* root){
    while (root->right != NULL)
    {
        root = root->right;
    }
    return root->data;
}
```

Output

```
Minimum element: 20
Maximum element: 80
```

(4)

```
int countNodes(Node* root) {  
    if (root == NULL)  
        return 0;  
    return 1 + countNodes(root->left) + countNodes(root->right);  
}
```

Output

```
Minimum value: 20  
Maximum value: 80  
Total number of nodes: 7
```